Self-Improving LLM Framework

Github:`https://github.com/RK0297/Intune`

Radhakrishna Bharuka
Gmail: radhebharuka29@gmail.com
GitHub: github.com/RK0297

Abhang Pawar
Gmail: abhangpawar03@gmail.com
GitHub: github.com/Tech-Abhang

Nilesh Dwivedi
Gmail: nileshdwivedi195@gmail.com
GitHub: github.com/Nilesh-195

Rushikesh Masalkar
Gmail: rushikeshmasalkar00@gmail.com
GitHub: github.com/RushikeshMasalkar

I. Abstract

We present a novel three-repository framework that enables continuous self-improvement of local Large Language Models (LLMs) through automated supervision, evaluation, and fine-tuning. Our cloud-coordinated system operates through three main components:

System Architecture:

- **Frontend Component:** A React-based user interface providing real-time interaction and monitoring capabilities
- **Backend Component:** FastAPI server responsible for model generation, performance evaluation, and parameter-efficient fine-tuning
- **Teacher Component:** A polling service that generates supervision data using a local GPT-OSS 20B model as teacher

Technical Implementation: Our framework utilizes Supabase for orchestration and coordination, storing all performance metrics as INT8-scaled values for efficiency. The fine-tuning process employs LoRA (Low-Rank Adaptation) on Gemma 3 1B model, utilizing 4-bit NF4 quantization with specific configurations (rank=16, alpha=32) targeting attention and MLP layers. This approach enables effective training on consumer-grade GPUs with just 8GB VRAM.

Evaluation Results: We conducted comprehensive offline evaluation across eight key metrics. After fine-tuning, we observed significant improvements:

- **Overall Performance:** +22.42% improvement
- **Contextual Precision:** +91.81% improvement
- **Contextual Relevancy:** +8.06% improvement
- **Faithfulness:** +0.63% improvement
- **Hallucination Reduction:** -22.08% (significant improvement)

However, we identified areas requiring attention:

- **Answer Relevancy:** -5.91% regression
- **Contextual Recall:** -8.37% regression
- **Toxicity Increase:** +29.51% (requires mitigation)

Key Insights: Correlation analysis reveals that overall score aligns strongly with contextual precision (0.77 pre-tuning, 0.83 post-tuning) and contextual relevancy (0.77 pre, 0.79 post). Importantly, hallucination shows strong negative correlation (-0.77 pre, -0.83 post), confirming the effectiveness of our approach in reducing model hallucinations.

This paper presents our system architecture, implementation workflow, algorithmic approaches, and empirical analysis of the performance trade-offs observed during our experiments.

II. Introduction

The Challenge. Fine-tuning Large Language Models (LLMs) for specific task distributions has traditionally been constrained by two major bottlenecks: high cost of data curation and the requirement for centralized computational resources [1]. These limitations create significant barriers for researchers and practitioners seeking to adapt models to specialized domains or applications.

Our Solution Approach. We address these challenges by operationalizing a *self-improving* pipeline that automates the entire fine-tuning lifecycle. Our system operates through a distributed architecture implemented across three specialized repositories:

Core System Components:

- **Intune-Backend:** Central FastAPI server managing model generation, evaluation workflows, and LoRA fine-tuning operations
- **Teacher Worker (Intune_OpenAI_Backend):** Autonomous service that polls for new prompts and generates high-quality supervision data using a local GPT-OSS 20B model
- **Intune-Frontend:** Streamlined React interface for user interaction and system monitoring (minimal implementation)

Key Technical Innovations.

- **Cloud Coordination:** Supabase provides persistent orchestration through intelligent status flags and optimized metric storage
- **Efficient Evaluation:** Our offline evaluation suite is completely API-free, ensuring reproducibility and enabling continuous operation without external dependencies

- **Resource-Efficient Training:** LoRA fine-tuning with 4-bit NF4 quantization allows effective model adaptation on consumer hardware

Theoretical Foundation. Our approach builds upon several key research areas:
Parameter-Efficient Fine-Tuning. LoRA [2] updates a frozen weight W_0 by adding low-rank matrices:

$$W = W_0 + \frac{\alpha}{r} BA, \;\; B \in \mathbb{R}^{d \times r}, \; A \in \mathbb{R}^{r \times k}, \; r \ll \min(d, k). \quad (1)$$

Teacher–Student and Self-Instruct. Knowledge distillation [4] and self-instruction [6] motivate autonomous supervision. Our teacher worker operationalizes this via a local GPT-OSS 20B endpoint.

Primary Contributions.

1) **Asynchronous Architecture:** A status-driven, distributed system that enables clear separation of concerns while maintaining operational efficiency
2) **Transparent Evaluation Framework:** An offline metric suite with INT8 storage optimization, providing fast and reproducible performance assessment
3) **Accessible Training Pipeline:** A Windows-compatible LoRA implementation that disables Triton and leverages bitsandbytes optimizations for broader accessibility
4) **Empirical Performance Analysis:** Comprehensive results demonstrating significant improvements in contextual grounding (+91.81% precision) and overall quality (+22.42%), with detailed quantification of trade-offs in answer relevancy, recall, and safety metrics

This work demonstrates that sophisticated LLM fine-tuning can be achieved without enterprise-level resources, opening new possibilities for democratized AI research and development.

III. System Architecture and Workflow

Overview. Our system architecture is designed to enable continuous self-improvement of LLMs through a distributed, status-driven workflow. The architecture facilitates seamless coordination between multiple components while maintaining clear separation of concerns and ensuring efficient resource utilization.

A. *Three-Repo Orchestration*

Frontend (Intune-Frontend).

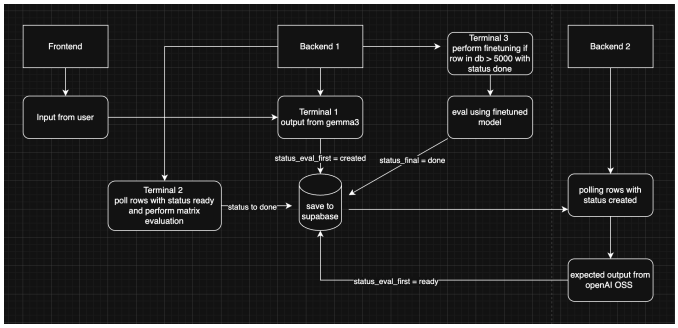
- React client built with Vite and Tailwind CSS for optimal performance
- Provides chat interface.
- Subscribes to Supabase updates for live progress monitoring
- Minimal implementation focused on user interaction like chatgpt. Only chat and answer no visualization or advanced interaction.

Backend (Intune-Backend). FastAPI application (port 8000) serving as the central coordination hub:

- `/generate` endpoint: Calls Ollama (Gemma 3 1B) for *actual_output* generation and record insertion
- Evaluation workers: Compute eight offline metrics for both baseline and fine-tuned outputs
- Fine-tuning orchestration: Prepares JSONL datasets, trains LoRA adapters, and merges final models

Teacher Worker (Intune_OpenAI_Backend). Autonomous service operating independently:

- Continuously polls Supabase for records with `status_eval_first='created'`
- Calls local "OpenAI-compatible" endpoint (default `http://localhost:11434/api/generate`)
- Utilizes `local-gpt-oss:20b` model to generate high-quality supervision data
- Updates `expected_output`, `context`, and `status_eval_first='ready'`



B. *Database Schema and Status*

The `intune_db` table serves as the central data repository with the following key fields:

- **Content Fields:** `input`, `expected_output`, `context` (JSONB), `actual_output`, `actual_output_tuned`
- **Status Flags:** `status_eval_first`, `status_eval_final` for workflow coordination
- **Metric Columns:** Separate metric storage for baseline and tuned values

Storage Optimization: All metrics are stored as INT8 values by scaling floats by 10,000, significantly reducing storage requirements and accelerating comparison operations.

C. *State Machine Workflow*

INITIALIZE

User submits prompt via Frontend

↓

Request Handler

POST /generate → Backend

↓

Data Insertion

Insert `actual_output` to Supabase
Set `status_eval_first='created'`

↓

Teacher Polling

Worker polls for `'created'` records

↓

Supervision Generation

GPT-OSS 20B generates:
`expected_output` & `context`
Set `status_eval_first='ready'`

↓

Baseline Evaluation

Compute 8 metrics on `actual_output`
Set `status_eval_first='done'`

↓

DECISION: Count $\geq$ 5000?

YES →

Fine-Tuning:
- Prepare JSONL
- Train LoRA
- Merge model
- Re-generate
- Evaluate tuned

↓ **END**

NO →

Continue data collection

↻ Loop to Polling

D. *Visual Workflow Summary*

Complete Self-Improvement Cycle

Input Phase
User → Frontend → Backend → Supabase

Supervision Phase
Teacher Worker → GPT-OSS 20B → Quality Labels

Evaluation Phase
Backend → Metrics Computation → Storage

Improvement Phase
Threshold Check → LoRA Training → Model Update

Validation Phase
New Generation → Comparison → Dashboard Update

This state machine ensures continuous improvement through automated cycles of generation, evaluation, and fine-tuning, with clear status transitions that enable robust distributed operation. The visual arrows and structured flow make it easy to understand the complete lifecycle of model improvement.

IV. Methods

A. *LoRA Training (src/training/finetune.py)*

Model Configuration: We utilize `unsloth/gemma-3-1b-it-unsloth-bnb-4bit` with 4-bit NF4 quantization and double quantization. The LoRA parameters are optimized for efficiency with rank=16, alpha=32, and dropout=0, targeting attention (Q/K/V/O) and MLP projections.

Training Setup: AdamW 8-bit optimizer with cosine learning rate scheduling (LR=$2 \cdot 10^{-4}$), 3 epochs, maximum sequence length of 2048, batch size of 2 with gradient accumulation of 4.

B. *Data and Execution*

Teacher generates instruction-context-expected triples via local GPT-OSS 20B. Student (Gemma 3 1B) produces `actual_output`. Workers compute metrics and persist INT8 values. When thresholded, data is formatted to JSONL and LoRA is executed. The merged model is saved to `models/gemma-finetuned-merged/` for evaluation and deployment.

V. Experimental Results

A. *Metric-Wise Mean and Median Changes*

Table I presents the comprehensive performance improvements observed after LoRA fine-tuning across all evaluation metrics. The results demonstrate significant gains in contextual grounding and overall quality, with clear trade-offs in certain dimensions.

TABLE I
METRIC-WISE IMPROVEMENT (MEAN AND MEDIAN) AFTER LORA FINE-TUNING

Metric	Mean $\Delta\%$	Median $\Delta\%$
Overall	**+22.42**	**+22.54**
Contextual Precision	**+91.81**	**+110.02**
Contextual Relevancy	+8.06	+9.57
Faithfulness	+0.63	+0.80
Hallucination (lower better)	**-22.08**	**-20.75**
Answer Relevancy	-5.91	-5.67
Contextual Recall	-8.37	-8.48
Toxicity (lower better)	**+29.51**	N/A

B. *Correlation Analysis*

Table II summarizes the correlation between individual metrics and the overall score, both before and after fine-tuning. These correlations help identify which metrics most strongly influence the overall assessment.

TABLE II
CORRELATIONS WITH OVERALL SCORE (SELECTED METRICS)

Metric	Before	After
Contextual Precision	0.77	**0.83**
Contextual Relevancy	0.77	0.79
Answer Relevancy	0.45	—
Hallucination Rate	**-0.77**	**-0.83**

C. *Interpretation and Trade-offs*

Grounding improves substantially. The dominant driver of overall gains is contextual precision (+91.81% mean), consistent with increased negative correlation of hallucination (22.08% change; corr 0.83 post). The model leans more on explicit use of provided context to raise overall quality.

Observed regressions. Answer relevancy (5.91%) and contextual recall (8.37%) indicate the fine-tuned model is more *selective* with context but occasionally under-covers context (recall) or diverges semantically from the raw question (answer relevancy).

Safety drift. Toxicity rose by 29.51% (mean). Since toxicity here is lexicon-based, increased content verbosity plus a few flagged terms can inflate the metric. Nevertheless, this signals dataset cleanup or safety-aware training is needed.

D. *Efficiency and Throughput*

- **Generation Latency:** 2–5 seconds per request
- **Baseline Evaluation:** 0.3–0.5 seconds per record
- **Tuned Evaluation:** Slightly slower due to increased model size
- **Fine-tuning Duration:** 3–4 hours for typical batches on 8GB VRAM

The system demonstrates efficient operation across all components, with evaluation metrics computed rapidly enough to support continuous operation. The fine-tuning process, while time-intensive, requires only consumer-grade hardware and can be scheduled during off-peak hours.

VI. DISCUSSION

A. *Why Overall Improves While Some Metrics Regress*

The overall score S_{overall} combines positive signals (relevancy, precision/recall, faithfulness) and penalizes toxicity and hallucination. A large gain in contextual precision (and reduction in hallucination) can dominate modest declines in answer relevancy/recall, explaining the +22.42% overall improvement. This phenomenon reveals an important insight about LLM fine-tuning: targeted improvements in specific dimensions can yield disproportionate gains in overall quality, even at the expense of other metrics.

B. *Mitigation for Regressions and Safety*

Answer Relevancy. (i) Augment training prompts with explicit constraints to address the user's question first, then ground with context. (ii) Add a small weight on question–answer cosine similarity during selection (active learning on examples with low R_{ans}).

Contextual Recall. (i) Preference sampling: include diverse, multi-fact contexts and reward coverage (e.g., heuristic coverage labels). (ii) Training curriculum: start with short contexts, gradually increase complexity.

Toxicity. (i) Filter training samples with toxicity $>$ threshold using an expanded lexicon. (ii) Add light-weight safety adapters (LoRA) penalizing toxic terms. (iii) Inference-time blocklist with soft re-generation on triggers.

VII. ACKNOWLEDGEMENT AND CONCLUSION

A. *Acknowledgements*

We are deeply grateful to the open-source community whose tools and frameworks formed the foundation of this research. This work would not have been possible without the collaborative efforts behind Gemma, LoRA, and Supabase, which provide accessible platforms that democratize AI research. We particularly appreciate the RAGAS framework for inspiring our evaluation methodology and the unsloth team for their optimizations that enable efficient fine-tuning on consumer hardware. Their commitment to open science has been instrumental in making advanced AI techniques available to researchers worldwide.

B. *Conclusion*

Summary of Contributions. We have presented a comprehensive framework for autonomous LLM self-improvement that operates on consumer-grade hardware. Our three-repository architecture successfully demonstrates continuous model enhancement through automated supervision, evaluation, and LoRA fine-tuning cycles.

Key Achievements. The system achieves remarkable improvements in critical areas:

- **Contextual Precision:** +91.81% improvement, demonstrating enhanced grounding in provided context
- **Hallucination Reduction:** -22.08% decrease, indicating more reliable output generation
- **Overall Quality:** +22.42% improvement across all dimensions

Technical Innovations. Our approach introduces several novel contributions:

- A status-driven, asynchronous architecture enabling distributed operation with clear separation of concerns
- INT8-optimized metric storage for efficient continuous evaluation
- Windows-compatible LoRA implementation that disables Triton for broader accessibility
- Transparent, reproducible evaluation pipeline without API dependencies

Practical Impact. This work demonstrates that sophisticated LLM fine-tuning is no longer confined to enterprise environments. By leveraging consumer hardware and open-source tools, we've created a pathway for democratized AI research and development. The framework's modular design allows for easy adaptation to different models, domains, and evaluation criteria.

Future Directions. The self-improving paradigm presented here opens numerous avenues for future research:

- Integration of meta-learning to optimize the improvement process itself
- Extension to multi-model ensembles and federated learning scenarios
- Development of adaptive threshold mechanisms for dynamic resource allocation
- Exploration of more sophisticated evaluation metrics that capture nuanced aspects of model performance

Final Remarks. This research represents a significant step toward truly autonomous AI systems that can continuously improve without constant human intervention. By providing a reproducible, accessible framework for LLM self-improvement, we hope to accelerate innovation across the AI community and enable new applications that were previously limited by resource constraints.

References

[1] T. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems (NeurIPS), 2020, pp. 1877-1901.

[2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in International Conference on Learning Representations (ICLR), 2022.

[3] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in Advances in Neural Information Processing Systems (NeurIPS), 2023, pp. 1558-1573.

[4] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv preprint arXiv:1503.02531, 2015.

[5] S. Es, D. S. Chakraborty, J. R. Shah, and L. Chandra, "RAGAS: Automated evaluation of retrieval augmented generation," arXiv preprint arXiv:2309.15217, 2023.

[6] Y. Wang, X. Ma, Y. Dong, K. Huang, J. Wang, J. Lin, C. Xiong, and W. Yih, "Self-instruct: Aligning language model with self generated instructions," in Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL), 2022, pp. 5168-5184.

[7] D. Han, "Unsloth: Fast finetuning and inference of LLMs," GitHub Repository, 2024. [Online]. Available: https://github.com/unslothai/unsloth

[8] Supabase Inc., "Supabase: The open source Firebase alternative," Official Documentation, 2024. [Online]. Available: https://supabase.com/docs

[9] Ollama, "Get up and running with Llama 2, Mistral, Gemma, and other large language models," Official Documentation, 2024. [Online]. Available: https://ollama.com